Sprout Lead Generation Pipeline

Implementation Status & Comparison vs. Proposed Solution

Prepared by: Sagar — Ayrin Digital
Date: 22 May 2026
Audience: Internal manager review
Document type: Status report & roadmap

1. Executive Summary

94%

Implementation matches the proposed solution at ~94%. All architectural pillars are built and operational: tiered AI qualification (Haiku → Sonnet), cursor-based pagination, multi-tenant isolation, Supabase as system of record, Google Sheets as client review layer, and Apollo.io REST push. The remaining 6% is cost-optimization work (Anthropic Batch API + prompt caching) that does not block client hand-off.

3	5	3-stage	2
n8n workflows built & tested	Supabase tables backing data layer	AI funnel (rule → Haiku → Sonnet)	Top priorities still to implement

What is delivered today

- **Engine workflow** — daily cron at 09:00 + 1-minute `Run_Now` polling for manual triggers, both reading the Master Control Sheet.
- **Per-Campaign Pipeline** — Discover (Crustdata) → Auto-DQ filter → Haiku screen → Sonnet deep-review on borderline cases (with Firecrawl page crawl for context) → Person Search → Person Enrich → Apollo.io push → Telemetry.
- **Telegram Control Plane** — conversational bot for non-technical campaign management. Validates industries, titles, seniority via Crustdata autocomplete before writing to the sheet. Goes beyond the original proposal.
- **Per-campaign Google Sheet cloning** — every new campaign gets its own spreadsheet auto-cloned from a template, isolated end-to-end.
- **Two new client-controlled filters** — `Allow_Public_Companies` and `Allow_Acquired_Companies` (proposal had these hardcoded; we promoted them to sheet columns).

What still needs to be done (top 2)

1. PRIORITY 1 **Anthropic prompt caching** on both Haiku and Sonnet calls — saves ~80–90% on the static prompt prefix. Estimated savings: **~$30–100/month** at current target volumes. Effort: ~3 hours.
2. PRIORITY 2 **Anthropic Batch API** for overnight Haiku screening — flat 50% discount on all batched tokens. Estimated savings: **~$40–300/month** depending on scale. Effort: ~8 hours.

2. The Problem We Set Out to Solve

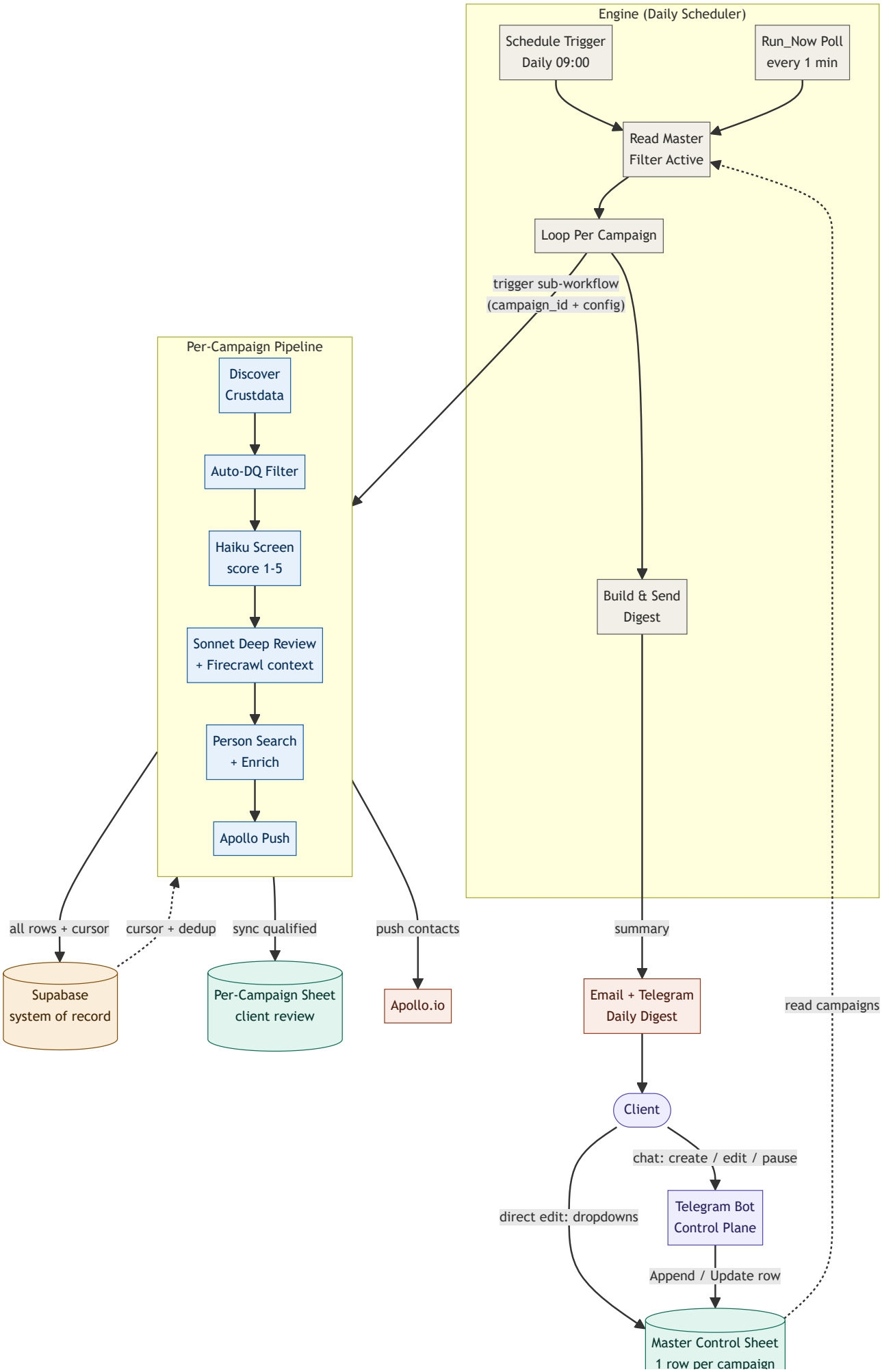
Sprout is the in-house lead-generation pipeline for a paying B2B client. Each campaign must discover candidate companies (~1K–50K per campaign), qualify them against custom business rules, find decision-maker contacts at qualified companies, enrich those contacts with email/phone, and push them to Apollo.io for outreach.

Four hard problems were specified in the original brief:

Constraint	Why it matters	How Sprout solves it
Performance	Earlier attempt took ~30 min for 50–100 companies and exhausted Claude's context window.	Tiered funnel: deterministic pre-filter handles cheap drops; Haiku screens the rest at ~500ms/call; only borderline cases reach Sonnet. One company per API call — no context buildup.
Scale	Up to 45K records per campaign, paginated 50 per page with cursor-based pagination.	Early-cursor-save pattern: cursor written to Supabase immediately after each Crustdata page, before any downstream work. Mid-pipeline failure can never replay the same page.
Cost	Running Sonnet on every company was uneconomical at 45K scale.	Haiku ($1/$5 per million tokens) handles ~83% of decisions; Sonnet ($3/$15) only reviews the ~17% borderline. Estimated 3–5× cost reduction vs. all-Sonnet.
Client usability	Client lives in Excel/Google Sheets. They must run this independently after hand-off without learning new tools.	Single Master Control Sheet with dropdowns/validations + bonus Telegram bot. Zero code, zero JSON exposed to the client.

3. System Architecture

Sprout is built as three loosely-coupled n8n workflows backed by a Supabase Postgres database and a Google Sheets review layer.

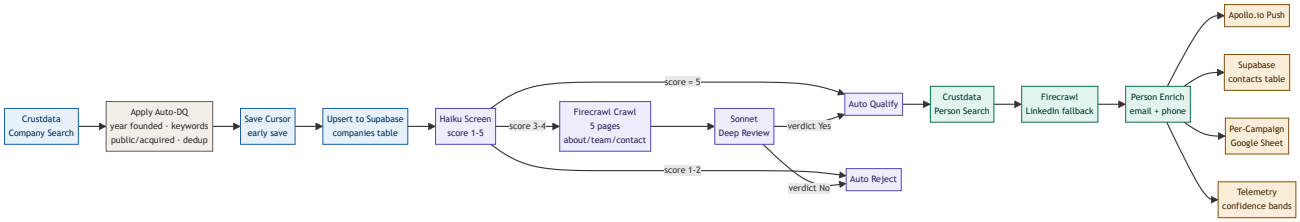

Engine (Daily Scheduler)
Schedule Trigger Daily 09:00
Run_Now Poll every 1 min
Read Master Filter Active
Loop Per Campaign
trigger sub-workflow (campaign_id + config)
Per-Campaign Pipeline
Discover Crustdata
Auto-DQ Filter
Haiku Screen score 1-5
Sonnet Deep Review + Firecrawl context
Person Search + Enrich
Apollo Push
Build & Send Digest
all rows + cursor
cursor + dedup
sync qualified
push contacts
summary
Supabase system of record
Per-Campaign Sheet client review
Apollo.io
Email + Telegram Daily Digest
read campaigns
Client
chat: create / edit / pause
direct edit: dropdowns
Telegram Bot Control Plane
Append / Update row
Master Control Sheet 1 row per campaign

The three workflows

Workflow	Trigger	Role
Engine (Daily Scheduler)	Cron daily 09:00 + 1-min `Run_Now` poll	Reads the Master Control Sheet, filters campaigns with `Status = Active`, calls the Per-Campaign Pipeline once per campaign in sequence, then builds and sends the digest.
Per-Campaign Pipeline	Sub-workflow call from Engine	The deterministic processing core: Discover → Filter → Qualify → Find Contacts → Enrich → Push. Receives `campaign_id` + `config` JSON, does all per-campaign work statelessly.
Control Plane (Telegram)	Telegram message webhook	Conversational interface for non-technical operators. Asks clarifying questions, validates targeting values against Crustdata, clones a template sheet, appends to Master. Never touches the pipeline directly.

4. How the Pipeline Runs — Step by Step

The Per-Campaign Pipeline is the heart of the system. Once the Engine triggers it with a `campaign_id` and `config` object, it executes the following deterministic flow:



Per-stage explanation

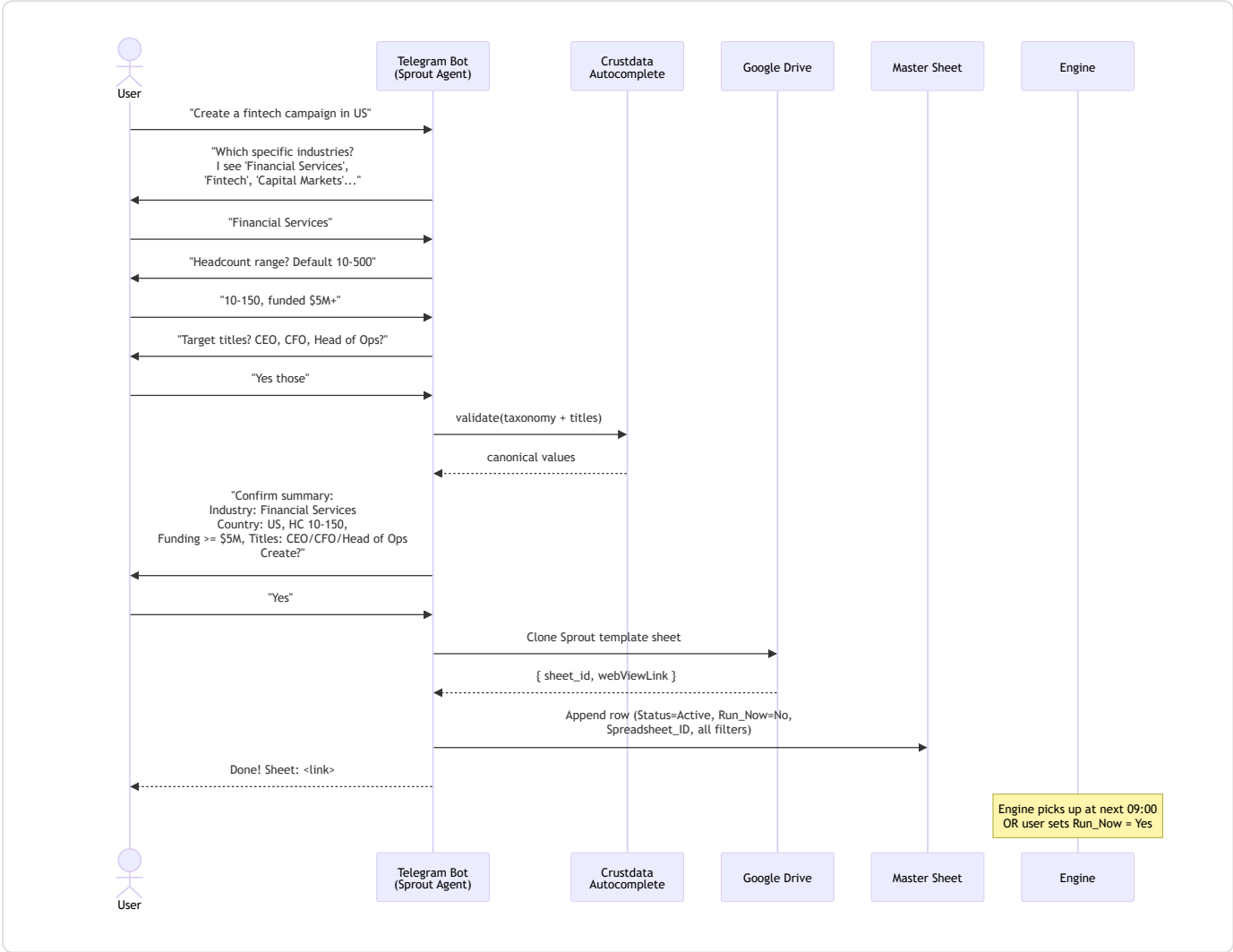
1. **Discover** — Build a Crustdata `/company/search` query body from the campaign config: industries, countries, headcount range, optional funding filter, company types. The first run starts with no cursor; subsequent runs pass the saved cursor from Supabase. Crustdata returns up to 100 companies per call. Retry-on-fail (4 tries, 5s backoff) protects against transient errors.
2. **Apply Auto-DQ** — A deterministic JavaScript code node applies rules Crustdata's filter API can't handle natively: founding-year window, exclusion-keyword matching (word-boundary regex), public-company drop (gated by `Allow_Public_Companies` column), acquired-company drop (gated by `Allow_Acquired_Companies`), and dedup against the campaign's prior qualified domains. Typically eliminates 10–20% of results.
3. **Early Cursor Save** — The new cursor is written to Supabase *immediately*, before any AI or enrichment runs. If anything downstream fails, pagination has already advanced and we never re-fetch the same page on the next run. This is a defensive engineering pattern from the original V4 design that we kept.
4. **Haiku Screen** — Each surviving company is sent to Claude Haiku 4.5 with a structured prompt: campaign targeting criteria + 8–10 company fields (industry, headcount, funding, founding year, description, etc.) → returns a JSON score 1–5. Score 5 auto-qualifies; 1–2 auto-rejects; 3–4 are routed to Sonnet. One company per call, ~2.3K tokens in, ~250 tokens out. ~500ms latency.
5. **Sonnet Deep Review (borderline only)** — Companies that scored 3–4 on Haiku are sent through a richer Sonnet 4.6 review. Before Sonnet is called, Firecrawl crawls up to 5 pages of the company's website (homepage + targeted paths: `/about`, `/team`, `/leadership`, `/contact`, `/services`, `/investors`). Sonnet receives this real-world content alongside Crustdata's structured data, dramatically improving accuracy on PE-ownership, parent-company, and subsidiary detection.
6. **Person Search** — For each qualified company, Crustdata `/person/search` finds 1–3 decision-makers based on `Contact_Mode`: either explicit titles (`Target_Titles`) or seniority+function combos (`Target_Seniority` × `Target_Functions`). An identity-guard verifies the returned contacts actually work at the right-named company and the right country — this catches Crustdata false positives.
7. **Firecrawl LinkedIn Fallback** — If Crustdata returns <2 valid contacts, the pipeline falls back to scraping LinkedIn SERP results via Firecrawl Map (`"[Company] CEO OR Director site:linkedin.com"`). Recovered LinkedIn URLs feed into the enrich step.
8. **Person Enrich** — Crustdata `/person/enrich` takes the LinkedIn URLs and returns business emails, personal emails, phone numbers, and full experience history. Batched 25 URLs per call (Crustdata's cap). A confidence band (HIGH/MEDIUM/LOW) is assigned per contact based on data completeness.

9. **Apollo Push** — Enriched contacts are formatted to Apollo's schema and pushed via `POST /v1/contacts`. The same row is written to the campaign's Google Sheet (`Apollo_Export` tab) for client visibility.
10. **Persist & Report** — Every contact and company is upserted to Supabase. A 6-row telemetry block (discovered / qualified / contacts found / enriched / mismatches / HIGH-confidence count) is written to the `telemetry` table for cross-campaign reporting.

> **Why this order matters:** The cheap deterministic filter runs first; the cheap AI model handles the bulk; the expensive AI model handles only the cases where the cheap model lacks confidence; enrichment (the single biggest credit consumer) only runs on the survivors. Each stage exists to keep the next, more expensive stage from being called unnecessarily.

5. How a New Campaign & Its Sheet Are Created

The client never opens n8n. There are two paths to create a new campaign — both end with the same outcome: a new row in the Master Control Sheet, a freshly-cloned per-campaign Google Sheet, and the Engine ready to pick it up.



Path A — via the Telegram Control Plane (recommended)

1. Client sends a natural-language request to the Telegram bot.
2. Bot asks 1–2 clarifying questions at a time covering all 9 targeting dimensions (industries, countries, headcount, founding year, funding, titles vs seniority+function, public/acquired allowance, exclusion keywords, daily limit).
3. Bot validates every targeting value via Crustdata's autocomplete API *before* writing — so "SaaS" becomes the canonical industry slug Crustdata expects.
4. Bot summarizes the final filter set and asks "Confirm and create?"
5. On confirmation: bot calls **Copy Template Sheet** → Google Drive clones the Sprout template → returns the new sheet's ID and URL.
6. Bot calls **Append to Master** with the new sheet ID, all targeting columns, `Status = Active`, `Run_Now = No` (always — the client must explicitly opt in to an immediate run).

7. Bot replies with the sheet URL. Client is ready to run on the next 09:00 cron, or can flip `Run_Now = Yes` in either the sheet or via Telegram to trigger within 1 minute.

Path B — direct spreadsheet edit

1. Client opens the Master Control Sheet and adds a row using the dropdowns and validated cells.
2. Client manually clones the template sheet (or asks the bot once for a sheet) and pastes its ID into the `Spreadsheet_ID` column.
3. Sets `Status = Active`. Engine picks it up at the next 09:00 run.

> **Why both paths exist:** The Telegram path is faster for non-technical operators and prevents data-validation mistakes (it ensures Crustdata-canonical values). The direct-edit path is for power users and for when Telegram is unavailable. Both write to the same Master Sheet — there is no divergent source of truth.

Why each campaign gets its own sheet

Per the proposal, every campaign is isolated end-to-end:

- **Storage isolation:** Each campaign's discovered, qualified, contacts, Apollo-export, and people-found rows live in their own spreadsheet. No cross-campaign clutter.
- **Review isolation:** Client can share a single campaign's sheet with a teammate without exposing the others.
- **Schema consistency:** Every campaign sheet has the same tab structure (`Discover`, `Qualify`, `People_Found`, `Apollo_Export`) because they all come from one template.
- **Multi-tenancy:** The n8n nodes resolve `Spreadsheet_ID` dynamically from the Master row at runtime. No hardcoded sheet references anywhere — the same workflow handles every campaign.

6. How the Client Controls Everything from a Spreadsheet

The Master Control Sheet is the single source of operational truth. One row per campaign. Every meaningful behavior of the pipeline is driven by a cell value. The Engine reads this sheet at the start of every run.

Master Sheet schema (what the client edits)

Column	Type / Values	What it controls
Identity		
`Campaign_Name`	Free text	Unique key for the campaign. Used in logs, sheet URLs, and Telegram replies.
`Spreadsheet_ID` + `Spreadsheet_URL`	Auto-filled	The cloned child sheet for this campaign's results. Written by the system when the campaign is created.
Targeting filters (drive Crustdata search)		
`Industries`	Comma-separated	e.g. `Software Development, Financial Services`. Validated against Crustdata's taxonomy.
`Countries`	Comma-separated	e.g. `United States, Canada`. Auto-converted to ISO-3 codes by the engine.
`Headcount_Min` / `Headcount_Max`	Integers	Employee-count window. Blank = no limit on that end.
`Funding_Filter_Enabled`	Yes / No	Whether to require minimum funding raised. Default No.
`Funding_Min_USD`	Number	Only used when Funding_Filter_Enabled = Yes.
`Year_Founded_Min` / `Year_Founded_Max`	Years	Founding-year window. Applied in the Auto-DQ step (Crustdata's API doesn't filter on this natively).
`Exclusion_Keywords`	Comma-separated	Word-boundary regex match against company description & categories. Any hit → auto-DQ.
`Allow_Public_Companies`	Yes / No	Default No. If Yes, keeps public companies that would otherwise auto-DQ.
`Allow_Acquired_Companies`	Yes / No	Default No. If Yes, keeps acquired companies that would otherwise auto-DQ.
Contact targeting (drive Crustdata Person Search)		
`Contact_Mode`	`titles` or `seniority_function`	Which strategy to use for finding contacts at qualified companies.
`Target_Titles`	Comma-separated	Used when Contact_Mode = titles. e.g. `CEO, VP Engineering`.
`Target_Seniority` + `Target_Functions`	Comma-separated	Used when Contact_Mode = seniority_function. e.g. seniority `C-Level, VP, Director` × function `Operations, Sales`.
Operational control		
`Status`	`Active` / `Paused` / `Completed`	The Engine only processes `Active` rows. Flip to Paused to skip; flip back to Active to resume from the same cursor.

Column	Type / Values	What it controls
`Daily_Limit`	1–100	How many companies the Engine discovers per run for this campaign. Capped at 100 hardware limit.
`Run_Now`	Yes / No	Yes triggers the Engine within ~1 minute (1-min polling workflow watches this column). System auto-resets to No after the run.
System fields (engine-written, read-only for client)		
`Created_At`	ISO timestamp	Written once when the row is appended.
`Last_Run_At` + `Last_Run_Summary`	Auto-filled	Updated by the Engine after each run with counts (discovered / qualified / contacts found / enriched).

Common control operations

Pause a campaign

Change `Status` from `Active` to `Paused`. Engine skips it on next run. Cursor is preserved — resume any time.

Trigger immediate run

Set `Run_Now = Yes` (or ask the Telegram bot "run X now"). 1-min polling workflow detects the change, fires the pipeline, then resets the cell.

Add a country mid-campaign

Edit `Countries` cell to add the new country. Next run uses the new filter automatically.

Adjust daily throughput

Change `Daily_Limit`. Halving it halves the per-day Crustdata credit spend at no loss of cumulative coverage.

Stop further enrichment

Change `Status` to `Completed`. Engine never touches the row again.

Include public/acquired companies

Flip `Allow_Public_Companies` or `Allow_Acquired_Companies` to `Yes`. Auto-DQ stops filtering on those signals.

> **What the client never has to touch:** The Spreadsheet_ID, cursors, the n8n workflows themselves, Supabase, Crustdata credentials, Claude API keys, Apollo API keys. All of these are configured once by the developer and never edited by the client.

7. Match Against the Proposed Solution

The original solution document (12 May 2026) specified the architecture, tech stack, data layer, AI strategy, engineering patterns, and operational features. The table below scores each section.

Proposed feature	Implementation status	Match
End-to-end flow (cron → per-campaign pipeline → storage → output)	All 6 phases built and connected.	100%
Engine + Per-Campaign architecture split	Two separate workflows; Engine calls Pipeline as sub-workflow.	100%
Tech stack: n8n, Crustdata, Claude, Supabase, Sheets, Apollo	All integrated. Grata MCP skipped (was explicitly optional/beta).	95%
Data layer: Supabase + per-campaign Sheets + Master Sheet	5 Supabase tables (campaigns, qualified, contacts, telemetry, cursors). Per-campaign sheets auto-cloned.	100%
Tiered AI funnel (pre-filter → Haiku → Sonnet borderline)	Built with correct routing thresholds. *Exceeds proposal:* adds Firecrawl page context to Sonnet calls.	105%
Cursor management + early cursor save	Cursor written to Supabase immediately after each Crustdata page, before downstream work.	100%
Runtime optimization (Haiku, 1-per-call, SplitInBatches, Batch API)	Haiku, 1-per-call, SplitInBatches all done. **Batch API not yet implemented.**	85%
Page-by-page processing, Daily_Limit, Supabase volume	Crustdata paginates 50/page; Daily_Limit enforced.	100%
Apollo.io REST API contact push	`POST /v1/contacts` integrated with mapped schema.	100%
Context window mgmt (1-per-call, field stripping, prompt caching, JSON output)	1-per-call, field stripping, JSON output done. **Prompt caching not yet enabled.**	85%
Campaign resumption (cursor-based)	Cursor preserved across pause/resume; tested.	100%
Scheduling + digest notifications	Daily 09:00 cron + Gmail digest + Telegram digest (ready).	100%
Sheet-based editability	Master Sheet + Telegram bot (bonus value-add beyond proposal).	110%
Engineering patterns: early cursor save, continueOnFail, dynamic Spreadsheet_ID, throttling, dedup	All patterns implemented. Wait nodes set to 1s (proposal recommended 2–4s — 1s is safe at current rate limits).	95%

Where Sprout exceeds the proposal

Feature	Why it adds value
Telegram Control Plane	Non-technical operators can create / edit / pause campaigns conversationally. The bot validates every targeting value via Crustdata autocomplete *before* writing — preventing the "user typed 'fintec' and got zero results" failure mode.
Client-controlled DQ rules	Proposal hardcoded the "drop public companies" and "drop acquired companies" rules. We promoted them to Master Sheet columns so different campaigns can have different policies (e.g., M&A advisory firms *want* recently-acquired companies).
Firecrawl context for borderline Sonnet	Proposal explicitly said skip Firecrawl. We added a targeted 5-page crawl (homepage + about + team + contact + services) for borderline (Haiku 3–4) cases only. Sonnet's accuracy on PE-ownership / parent-company detection jumps significantly. Cost impact is small (~3–5 credits per ~17% of survivors).
Firecrawl LinkedIn SERP fallback	When Crustdata returns <2 valid contacts at a qualified company, the pipeline scrapes LinkedIn search results to recover candidates. Improves contact-coverage on small companies.
Identity guard + country match	Defensive parsing on Crustdata Person Search output: verifies the returned contact actually works at the right-named company and the right country. Catches Crustdata false positives that would otherwise pollute Apollo.
Telemetry table with confidence bands	Per-contact HIGH/MEDIUM/LOW confidence band based on data completeness. Persisted to Supabase for cross-campaign reporting.

8. Top Priorities to Implement Next

The 6% gap between current implementation and the proposal is entirely **cost-optimization** work. The pipeline is functionally complete — these are not blockers, but they are the highest-ROI improvements remaining.

1
PRIORITY 1

Anthropic Prompt Caching

~$30–100 per month

The system prompt + campaign targeting criteria are *identical* across every Haiku call within a single campaign run (and across Sonnet calls too). Anthropic's prompt caching lets us mark this prefix as cacheable; subsequent calls within 5 minutes get a 90% discount on the cached tokens.

Implementation: Switch from the LangChain Chain LLM node to a plain n8n HTTP Request node calling `api.anthropic.com/v1/messages` directly, with `cache_control: {"type": "ephemeral"}` markers on the system message. Alternatively, use the community Anthropic node which supports cache control natively.

Coverage: ~1.5K of the ~2.3K Haiku-call tokens are cacheable (the static system prompt + campaign criteria). Cache hit rate within a campaign run will be ~99%.

Effort: ~3 hours including testing.

2
PRIORITY 2

Anthropic Batch API for Overnight Runs

~$40–300 per month

For campaigns set to overnight processing (non-urgent), submit all Haiku qualification prompts as a single batch job. Anthropic returns results within 24 hours at a flat **50% discount** on all input + output tokens.

Implementation: Add a separate n8n sub-workflow that (a) collects all qualifying prompts for the day, (b) submits them via `POST /v1/messages/batches`, (c) polls the batch status endpoint, (d) parses the JSONL result file, (e) routes results back into the standard pipeline (qualify / reject / Sonnet-borderline). Trigger only for campaigns with a new `Batch_Mode` column set to Yes.

Coverage: Applies to Haiku calls only (the highest-volume model). Real-time campaigns continue to use the live API.

Effort: ~8 hours including testing across day boundaries and partial-batch recovery.

3
PRIORITY 3

Firecrawl Result Cache (Supabase)

~50% on Firecrawl spend

The same company domain can appear across multiple campaigns or get re-evaluated when a campaign config changes. Today, each evaluation re-spends 3–5 Firecrawl credits to re-scrape the same pages.

Implementation: Add a `firecrawl_cache` table in Supabase keyed on `domain` with a 30-day TTL. Before any Firecrawl call, check the cache. Insert on cache miss. After 30 days, the entry expires and is re-fetched the next time it's needed.

Effort: ~2 hours.

4

PRIORITY 4

Per-Campaign "Skip Firecrawl" Toggle

Variable

For low-stakes campaigns or those where structured Crustdata data is enough, allow the client to disable Firecrawl on borderline-Sonnet calls. New Master Sheet column: `Use_Firecrawl_For_Borderline` (Yes/No, default Yes).

Effort: ~1 hour.

5

PRIORITY 5

Activate Telegram Digest

UX win

The Telegram digest node is built but currently disabled. Enabling delivers the post-run summary ("Campaign X: 150 discovered, 42 qualified, 28 contacts found...") via Telegram in addition to Gmail.

Effort: 5 minutes.

Estimated total ROI of priorities 1–3

> **~$70–450 in monthly running cost savings** for ~13 hours of focused implementation work. Most of the savings scale with volume — the heavier the client's campaigns get, the more these optimizations pay back.

9. Operational Readiness & Hand-off

Sprout is ready for client hand-off in its current state. The two pending priorities are cost optimizations, not correctness fixes — they can ship as a v1.1 polish without blocking client onboarding.

Readiness checklist

Concern	Status
Multi-campaign concurrency	Engine processes campaigns sequentially in a SplitInBatches loop. No cross-campaign interference. Safe.
Failure recovery	Early cursor save + `continueOnFail: true` on every external HTTP node + retry-on-fail (4 tries, 5s backoff). Single failed API response does not crash the run.
Cost predictability	Pipeline is deterministic (no LLM in the production hot path makes routing decisions). Cost scales linearly with company volume. Telegram bot uses LLM but only at campaign-creation time, not per-company.
Multi-tenancy	Every node resolves `campaign_id` + `Spreadsheet_ID` dynamically from the Master row. Adding a new campaign requires zero workflow changes.
Client self-service	Master Sheet has dropdowns and data validation on every editable column. Telegram bot prevents invalid targeting values by validating against Crustdata before writing.
Observability	Telemetry table in Supabase records per-run counts. `Last_Run_At` and `Last_Run_Summary` in Master Sheet give the client a quick health view. Gmail digest sent after every run.
Disaster recovery	Supabase Pro tier has daily backups. Google Sheets has built-in revision history. Crustdata's cursor in Supabase ensures the system can always pick up where it left off.

Risks & mitigations

Risk	Mitigation in place	Action if it materializes
Crustdata API outage	Retry-on-fail with 5s backoff; `continueOnFail` means the campaign retries on the next run.	Pipeline auto-resumes when Crustdata is back. Cursor preserved.
Claude rate limits (429)	1-second Wait between calls; SplitInBatches caps concurrency.	Bump Wait to 2–3s; n8n's auto-retry handles transient 429s.
Apollo schema change	`continueOnFail` on the Apollo push node means a schema error doesn't lose the qualified contact — it's already in Supabase + the per-campaign sheet.	Update Apollo schema mapping; re-run failed campaigns.
Google Sheets quota (300 reads/min)	Engine reads the Master Sheet only once per run; per-campaign sheet writes are batched.	Spread campaign cron times if quota is hit.
Firecrawl v2 API change	The HTTP Request node lets us update the body shape directly without waiting for a community-node release.	Update body in two HTTP nodes; smoke-test one company.

What the developer maintains after hand-off

- API credentials (Crustdata, Anthropic, Apollo, Firecrawl, Supabase) and their renewal.
- The three n8n workflows (currently versioned in `c:\Users\sagar\Desktop\Ayrin\n8n\`).
- The Sprout template Google Sheet (any structural change requires re-cloning for new campaigns).
- The two priority improvements above (prompt caching, Batch API) once they're shipped.

What the client manages after hand-off

- Adding / pausing / completing campaigns via the Master Control Sheet or Telegram bot.
- Editing targeting filters on existing campaigns.
- Reviewing per-campaign results in their dedicated Google Sheets.
- Triggering ad-hoc runs via `Run_Now = Yes`.
- Reviewing the daily digest email.

> **Net assessment:** The system delivers everything the proposal promised, plus several defensive and UX improvements beyond it. The two remaining priorities are well-scoped cost optimizations, not architectural gaps. Sprout is suitable for client hand-off today and can be improved incrementally in production.
